

Architecture

Gen AI Platform for Automated Content Transformation · SIH 2026 · PS 26154 · NTRO

1. Problem

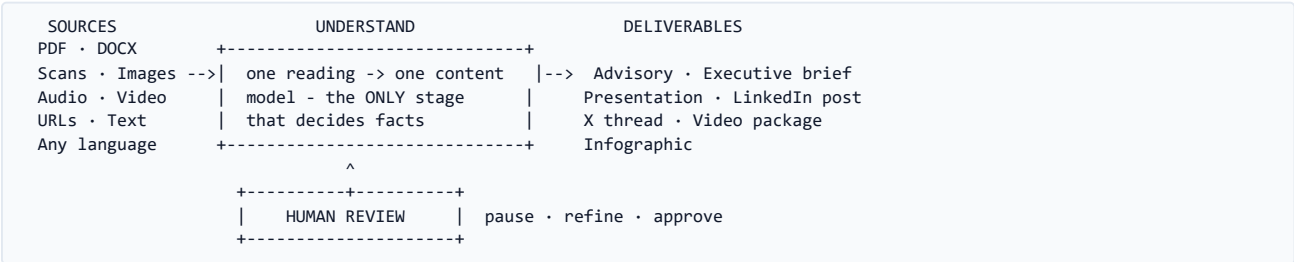
One source document must become many deliverables — a structured advisory, an executive briefing, a presentation, a LinkedIn post, an X thread, a video package, an infographic. Today that is manual: slow, inconsistent, and demanding expertise in both the domain and each channel's conventions.

The obvious implementation — prompt the model once per output — fails in four specific ways:

Failure	Cause
Fact drift	each call re-reads the source independently, so outputs disagree with each other
Inconsistency	seven artefacts become seven independent interpretations
No traceability	output is prose; a claim cannot be traced to the passage behind it
Cost scales multiplicatively	the full source is re-sent once per output

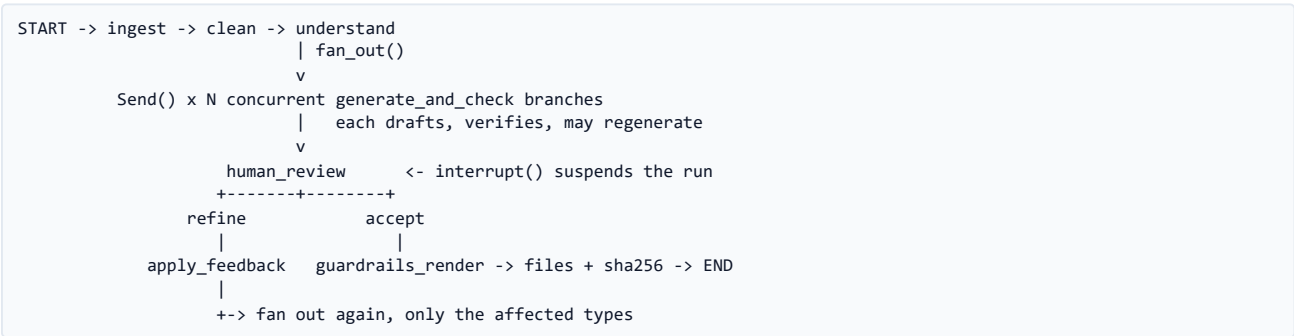
2. The core design decision

Separate *understanding* from *expression*. Read the source **once**, freeze its meaning into a single structured **content model**, and make every artefact a **projection of that object** rather than a fresh reading.



Because there is exactly **one place a fact can be decided**, artefacts cannot contradict each other, and adding an eighth output costs one small call rather than another full read.

3. Execution — a LangGraph state machine



Every node is a plain Python function over one shared `EngineState`. **LangGraph — not the model — decides ordering, concurrency, pausing and looping.** Five mechanisms carry the design:

- ① **Fan-out via `Send`.** A conditional edge returning `[Send(node, payload)]` schedules one concurrent task per selected output. Measured: three branches whose durations sum to 8.2 s complete in 6.6 s wall-clock. A `Send` payload *replaces* branch state, so it carries `source_input`, `corpus`, `content_model` and `selected_outputs` explicitly.
- ② **Reducers.** Three branches write `artefacts` at once. `Annotated[dict, merge_dicts]` makes LangGraph merge concurrent writes instead of letting the last one win — without it, two of three artefacts are silently lost.
- ③ **Structural human review.** `human_review` calls `interrupt(payload)`, which genuinely stops execution; the checkpointer persists state so `Command(resume={...})` continues it. Review is not a UI overlay — the graph halts.
- ④ **Verification feeds generation, rather than reporting after it.** Each branch tests every draft sentence against the source through three independent checks — stopword-filtered token overlap, explicit number/date/version comparison against the whole corpus, and IOC pattern membership. Below a 0.6 support ratio the branch regenerates **once**, with the offending claims named in the prompt, and keeps the rewrite only if it scores strictly better.
- ⑤ **Deterministic rendering.** The model emits *blueprints* (structured JSON); local libraries emit files (`python-pptx`, `ReportLab`, `Pillow`). Nothing about a file is model-generated, so rendering is reproducible and every artefact carries a meaningful sha256.

4. Data contract

```
class EngineState(TypedDict):
    source_input: dict          # sources + generation parameters
    corpus: list[dict]          # [{"text", "source_ref", "kind", "source_id"}]
    content_model: dict         # the single source of truth
    artefacts: Annotated[dict, merge_dicts] # type -> draft
    fact_checks: Annotated[dict, merge_dicts] # type -> verdicts + meta
    review: dict | None
    export: dict | None         # validated files + sha256 manifest
```

Every corpus block carries a `source_ref` from the moment it is parsed (`report.pdf p.2 ¶3`). References are never invented downstream — that is what makes citation tracing possible at all.

5. Components

Module	Responsibility
graph.py · state.py	Graph wiring, conditional edges, fan-out, interrupt; the state contract
ingestion/	Docling parsing, PyMuPDF fast path, vision OCR for thin pages and embedded figures, audio transcription, language detection and translation, reference-preserving chunking
understand.py	Corpus → content model; map-reduce batching above the context window; fact grounding; deterministic IOC extraction
generate.py	Drafting, JSON-schema enforcement, field repair, three verification checks, bounded regeneration
render.py · recipes.py	PDF/PPTX/PNG/SRT renderers and design templates; schemas, prompts, artefact profiles
providers.py · api.py	Provider rotation; FastAPI layer and run registry

Ingestion in one line: a *thin* text layer is not the same as an *empty* one — pages under 250 characters are rendered and transcribed by a vision model, which recovered **11,406 characters** from a document that previously yielded 17. Vision calls are batched four-per-request, taking a 27-image document from ~27 requests to ~7 against a per-minute quota.

6. Provider abstraction — the deployment answer

Every model call passes through one `LLMProvider` interface. Four backends implement it — Groq, OpenRouter, Gemini, Ollama — and a rotating provider fails over **per call**, with cooldowns for quota and capacity errors and capability flags so a vision call never routes to a text-only backend.

```
LLM_FALLBACK_CHAIN=groq,openrouter,gemini # cloud free tiers, with failover
LLM_FALLBACK_CHAIN=ollama                 # fully offline, inside the network
USE_STUB=1                                # no model at all
```

One configuration line, no code change. With `DOCLING_OCR=1` the parsing path is local too, giving an air-gapped configuration.

7. Verified behaviour

82 offline tests (no network, no API key) plus end-to-end runs on real material: an 803-page bilingual document, handwritten lecture notes, an image-only PDF, 67-slide decks, Hindi audio and live URLs. A representative run produces a content model, three artefacts with per-claim verdicts, and an export pack of PDF, PPTX and PNG files — in roughly 45–90 seconds.

8. Known limits

Stated plainly, because an intelligence platform should not overstate itself. Fact-checking is **lexical and numeric**, not semantic entailment — a well-paraphrased fabrication would pass. PDF and poster rendering is **Latin-script only**, because Pillow and ReportLab do not shape complex scripts; non-Latin output works in `.txt` and `.pptx`, and the affected options are disabled in the interface with a warning rather than emitting empty boxes. The checkpointer is **in-memory**, so run history does not survive a restart; `SqliteSaver` is the production path.